AWS NETWORKING LAB REPORT

Connecting to a Private EC2 Instance via a Public EC2 Instance (Bastion Host)

Amazon Web Services — VPC, Subnets, Internet Gateway, Route Tables & EC2

Cloud Platform	Amazon Web Services (AWS)
Region	US East (N. Virginia) — us-east-1
VPC Name	my-vpc (192.168.1.0/24)
Public Subnet	Public-Subnet — 192.168.1.0/25
Private Subnet	private-subnet — 192.168.1.128/25
Instances	Public-EC2 (public subnet) and Private-EC2 (private subnet)
Operating System	Ubuntu Server 26.04 LTS
Objective	Securely access a private-subnet EC2 instance using a public-subnet EC2 instance as a bastion/jump host

1. Objective

The objective of this lab is to design a secure two-tier network architecture on AWS and demonstrate how an EC2 instance placed in a private subnet — which has no direct route to the internet and no public IP address — can be accessed securely using a second EC2 instance placed in a public subnet as an intermediate "jump box" or bastion host.

This exercise covers the creation of a custom Virtual Private Cloud (VPC), public and private subnets, an Internet Gateway, route tables, security groups, EC2 instance provisioning, and SSH key-pair based remote access.

2. Theory

2.1 Virtual Private Cloud (VPC)

A Virtual Private Cloud (VPC) is a logically isolated section of the AWS cloud in which a user can launch AWS resources in a virtual network that they define. A VPC closely resembles a traditional network that an organization would operate in its own data center, but with the benefits of using the scalable infrastructure of AWS. Every VPC is defined by a range of IP addresses expressed in CIDR notation (e.g., 192.168.1.0/24), and can be divided into smaller sub-networks called subnets.

2.2 Subnets — Public vs. Private

A subnet is a range of IP addresses within a VPC that is tied to a specific Availability Zone. Subnets are classified as:

- Public Subnet: A subnet whose associated route table has a route to an Internet Gateway. Instances in a public subnet can have a public IP address and can communicate directly with the internet.
- Private Subnet: A subnet whose route table does not have a direct route to an Internet Gateway. Instances here cannot be reached from, or reach out to, the internet directly, which makes them ideal for hosting databases, application servers, or any resource that should not be exposed publicly.

2.3 Internet Gateway (IGW)

An Internet Gateway is a horizontally scaled, redundant VPC component that allows communication between instances in a VPC and the internet. It serves two purposes: providing a target in VPC route tables for internet-routable traffic, and performing network address translation (NAT) for instances that have been assigned public IPv4 addresses. An IGW must be created and then attached to a VPC before it can be used.

2.4 Route Tables

A route table contains a set of rules, called routes, that determine where network traffic from a subnet or gateway is directed. Each subnet in a VPC must be associated with a route table. A route table entry of destination 0.0.0.0/0 pointing to an Internet Gateway makes the associated subnet "public." A subnet associated with a route table that lacks such a route remains "private."

2.5 Security Groups

A security group acts as a virtual, stateful firewall for an EC2 instance, controlling inbound and outbound traffic at the instance level. Rules are defined by protocol, port range, and source/destination. In this lab, the private instance's security group is configured to accept SSH (port 22) traffic only from the public instance, not from the internet at large.

2.6 EC2 Instances and Key Pairs

Amazon Elastic Compute Cloud (EC2) provides resizable virtual machines called instances. Secure shell (SSH) access to a Linux instance is authenticated using a public/private key pair: AWS stores the public key on the instance at launch, while the user retains the private key (.pem file) needed to prove their identity when connecting.

2.7 Bastion Host (Jump Server) Concept

A bastion host is a special-purpose instance placed in a public subnet that is intentionally exposed to reach an otherwise unreachable network segment. Administrators first connect to the bastion host over SSH, and from there "hop" to instances in the private subnet using their private IP addresses. This pattern minimizes the attack surface, since only one hardened, closely-monitored host is internet-facing, while all other servers stay fully isolated from direct internet access.

2.8 How the Connection Flow Works

The overall flow implemented in this lab is as follows:

- The administrator's local machine connects over SSH to the Public-EC2 instance using its public IP address and a private key (test1.pem).
- The same private key is securely copied (via SCP) from the local machine onto the Public-EC2 instance.
- From inside the Public-EC2 instance, the administrator runs SSH again — this time targeting the private IP address of Private-EC2 — using the copied key.
- Because both instances reside within the same VPC, the Public-EC2 instance can route directly to the private subnet, granting secure access to Private-EC2 without ever exposing it to the internet.

3. Network Architecture Summary

VPC CIDR	192.168.1.0/24 (my-vpc)
Public Subnet CIDR	192.168.1.0/25 (Public-Subnet, us-east-1a)
Private Subnet CIDR	192.168.1.128/25 (private-subnet, us-east-1a)
Internet Gateway	MyIGW — attached to my-vpc
Public Route Table	public-RT → 192.168.1.0/24 = local, 0.0.0.0/0 = MyIGW; associated with Public-Subnet
Private Route Table	Private-RT → 192.168.1.0/24 = local only (no internet route); associated with private-subnet
Public-EC2	Ubuntu instance in Public-Subnet with a public IP — acts as the bastion/jump host
Private-EC2	Ubuntu instance in private-subnet, private IP 192.168.1.191, no public IP

With this setup, only Public-EC2 is internet-reachable. Private-EC2 is completely shielded from the internet and can only be reached from within the VPC — in this case, via Public-EC2.

4. Step-by-Step Implementation

Step 1: Create the VPC

From the AWS Management Console, navigate to VPC → Your VPCs → Create VPC. Select "VPC only," assign the name tag my-vpc, and specify the IPv4 CIDR block 192.168.1.0/24. Leave IPv6 disabled and create the VPC.

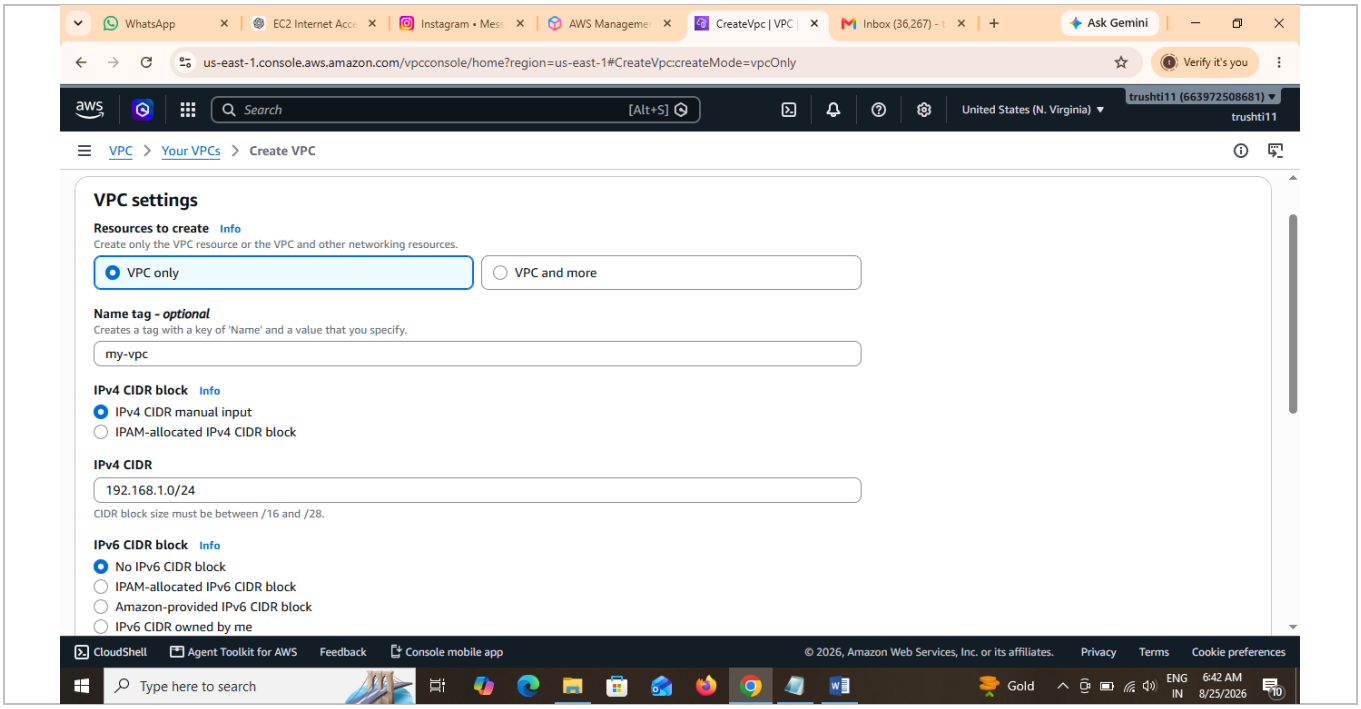

Figure 1: Creating the VPC (my-vpc) with CIDR block 192.168.1.0/24.

Step 2: Create Public and Private Subnets

Within my-vpc, navigate to Subnets → Create subnet and add two subnets in the us-east-1a Availability Zone:

- Public-Subnet — CIDR 192.168.1.0/25 (128 IPs)
- private-subnet — CIDR 192.168.1.128/25 (128 IPs)

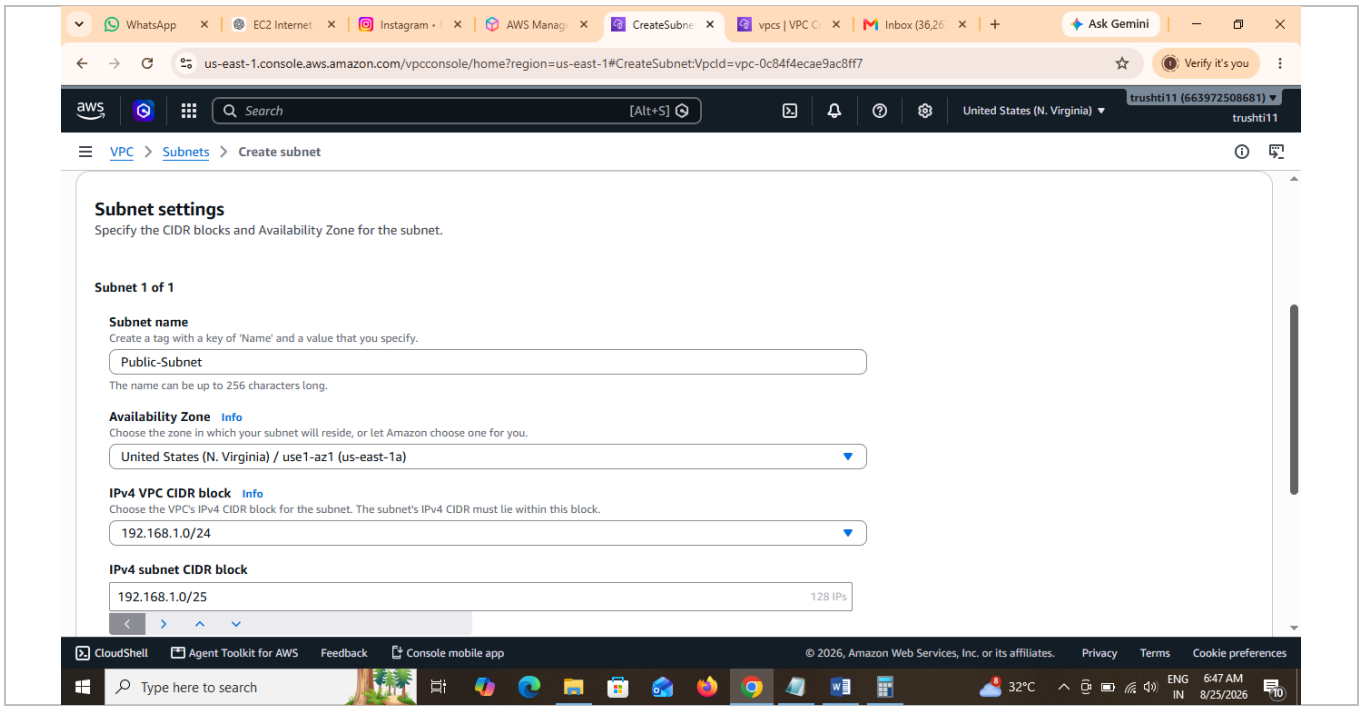

Figure 2: Creating Public-Subnet with CIDR 192.168.1.0/25.

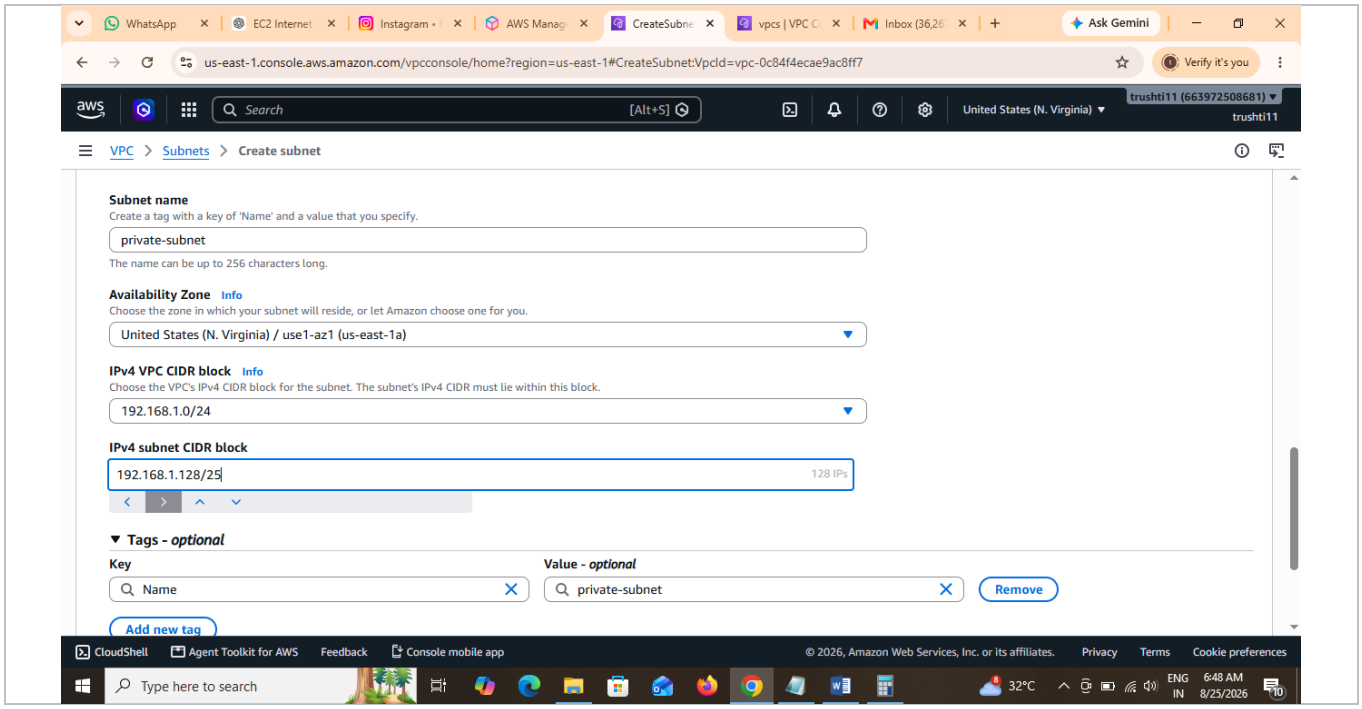

Figure 3: Creating private-subnet with CIDR 192.168.1.128/25.

Step 3: Launch the Private EC2 Instance

Go to EC2 → Instances → Launch an instance. Name the instance PrivateEC2 and choose the Ubuntu Server 26.04 LTS AMI with a t3.small instance type.

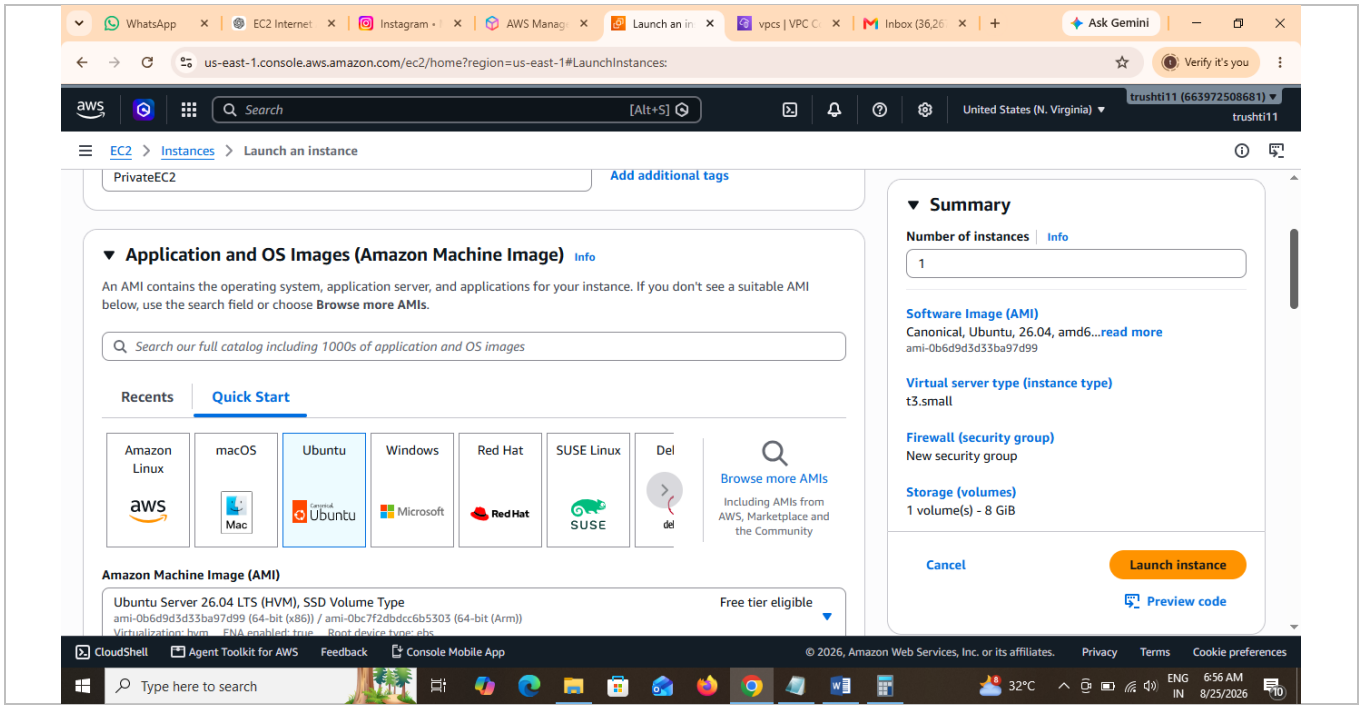

Figure 4: Naming the instance and selecting the Ubuntu 26.04 LTS AMI.

Select an existing key pair (test1) for SSH authentication.

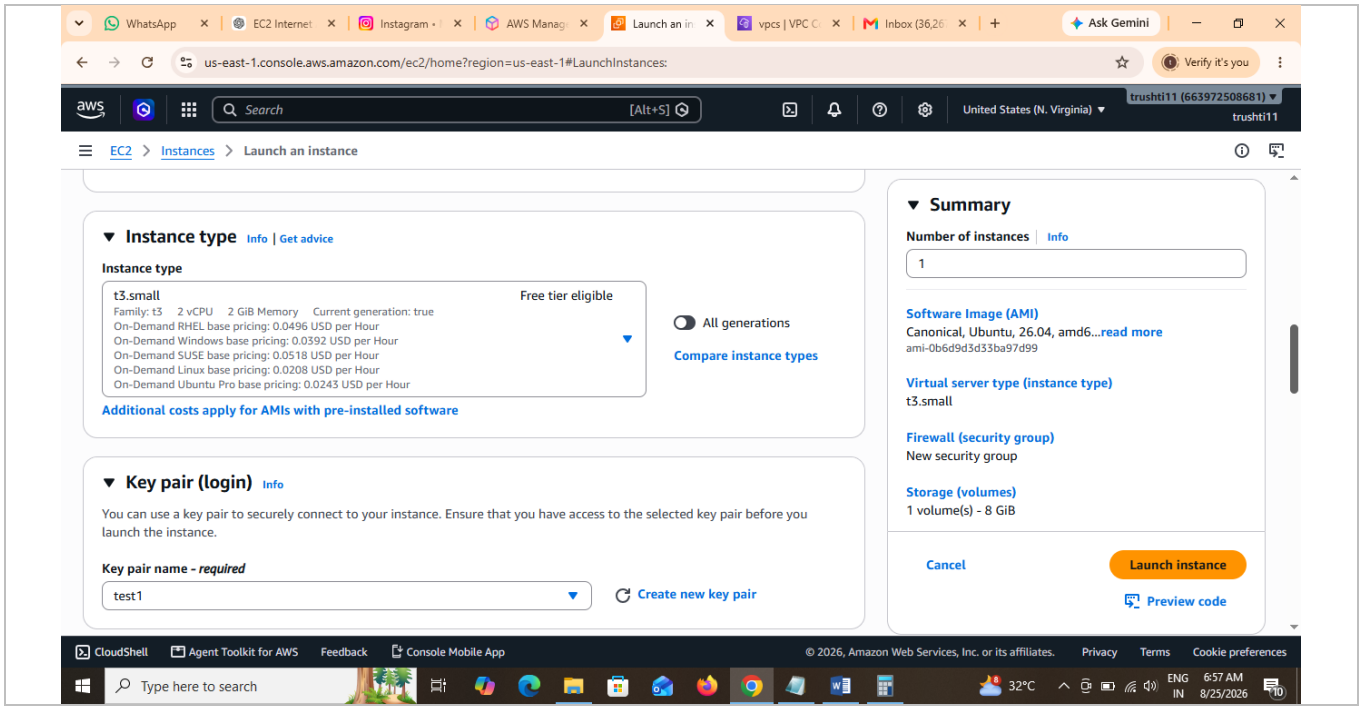

Figure 5: Choosing instance type t3.small and key pair test1.

Under Network settings, choose my-vpc, select private-subnet, and — critically — set "Auto-assign public IP" to Disable, so the instance never receives a public IP address. A new security group is created to control inbound access.

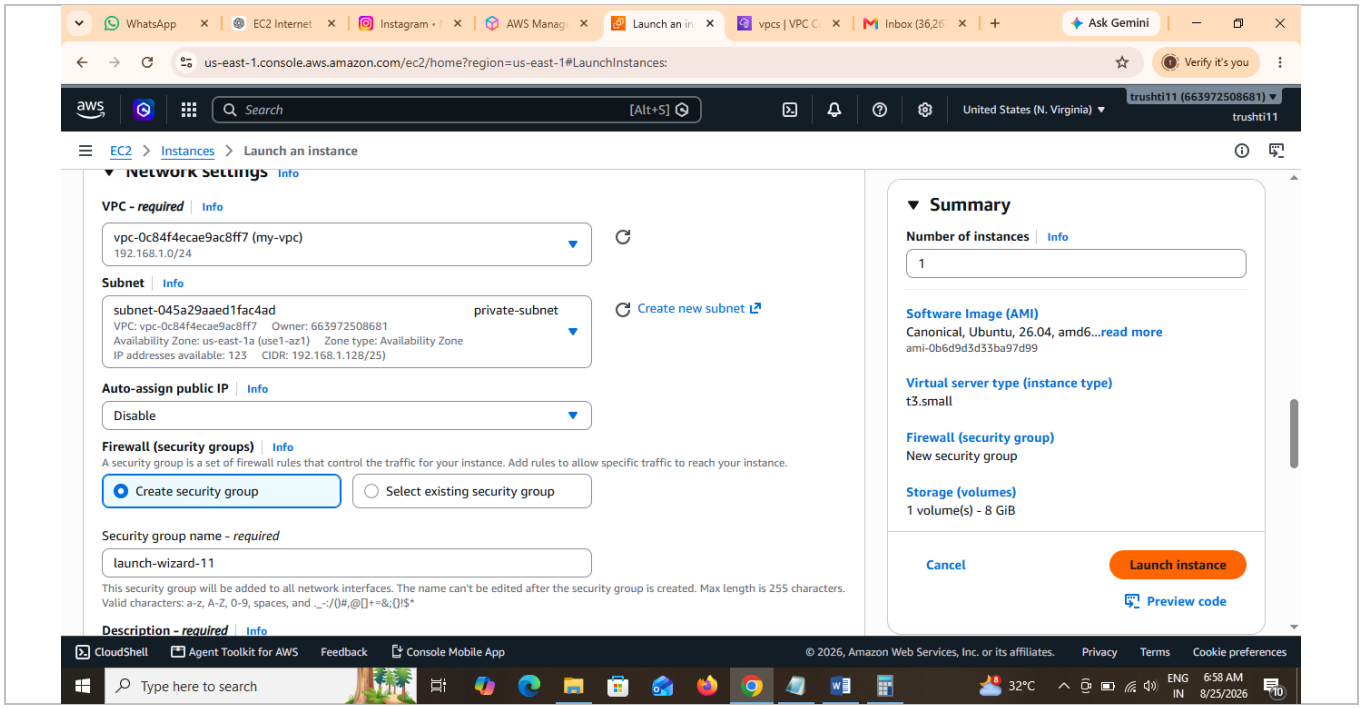

Figure 6: Placing the instance in private-subnet with public IP assignment disabled.

Step 4: Create and Attach an Internet Gateway

Navigate to VPC → Internet Gateways → Create internet gateway, name it MyIGW, and create it.

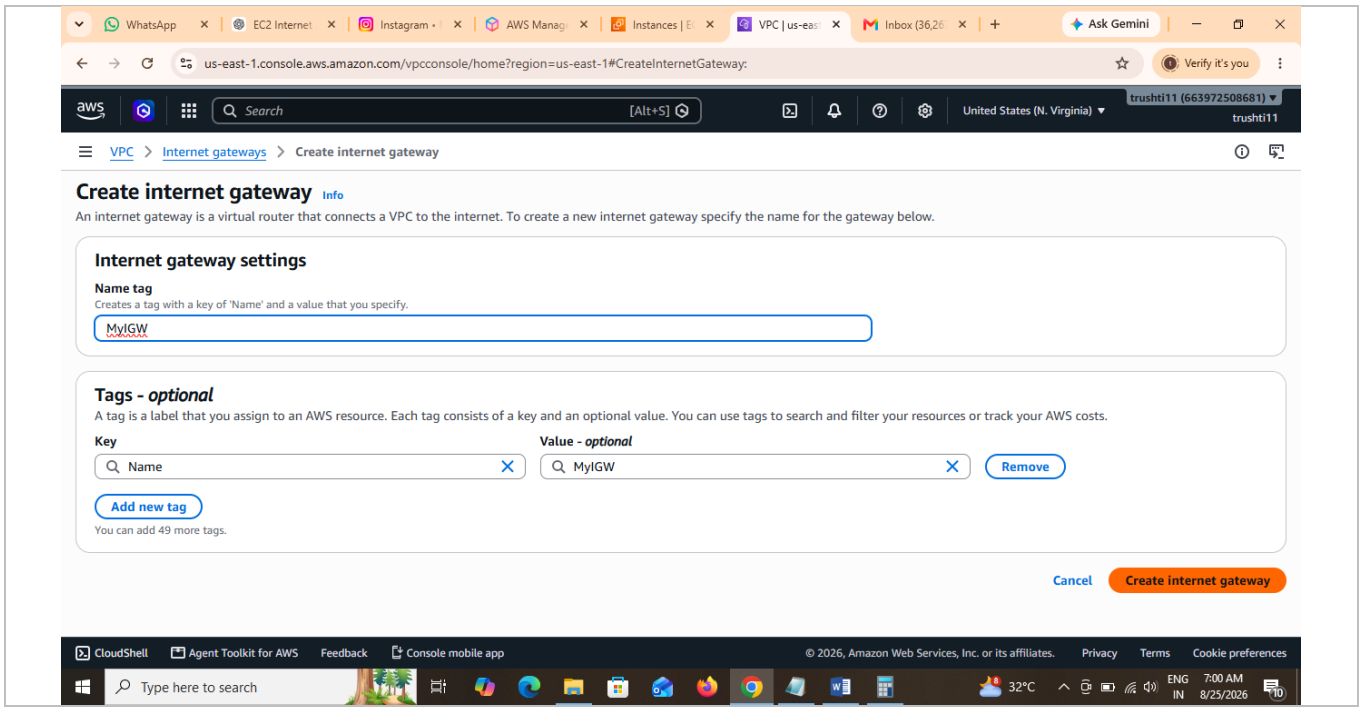

Figure 7: Creating the internet gateway MyIGW.

Once created, select MyIGW and choose Actions → Attach to VPC.

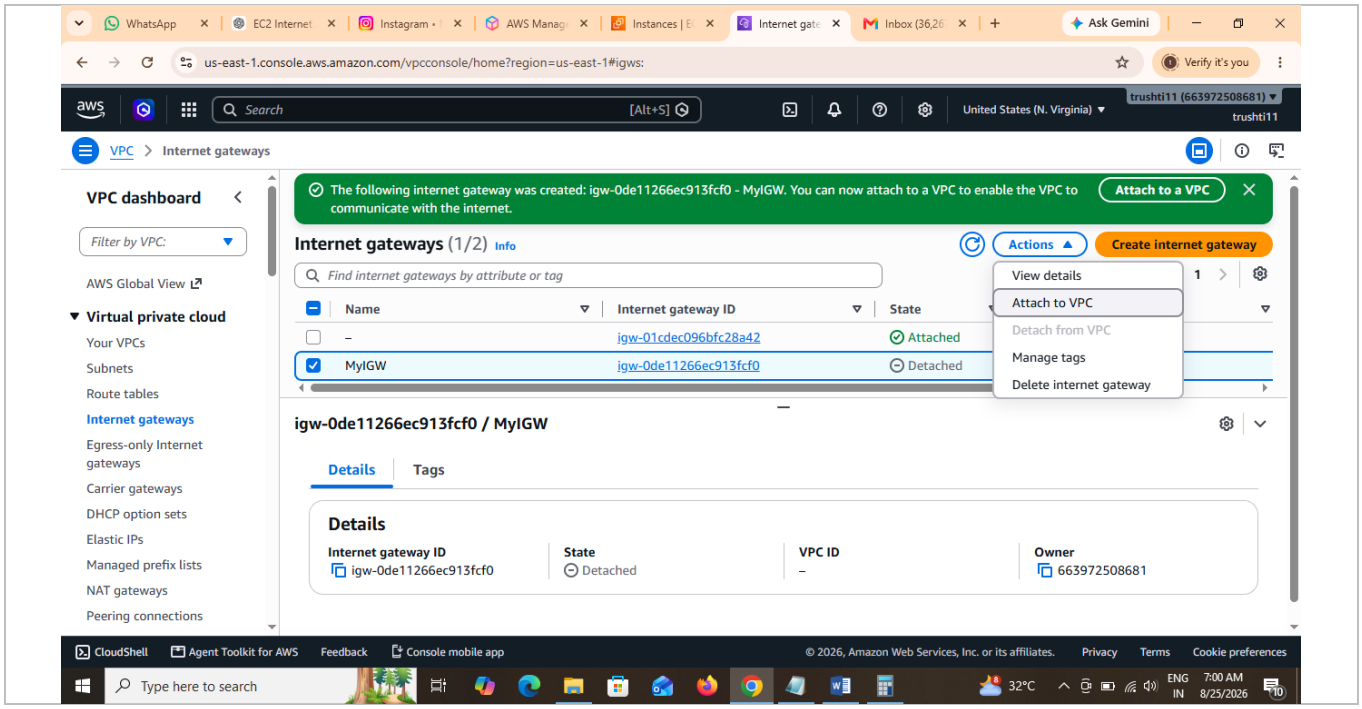

Figure 8: Attaching MyIGW to a VPC from the Actions menu.

Select my-vpc as the target VPC and confirm the attachment.

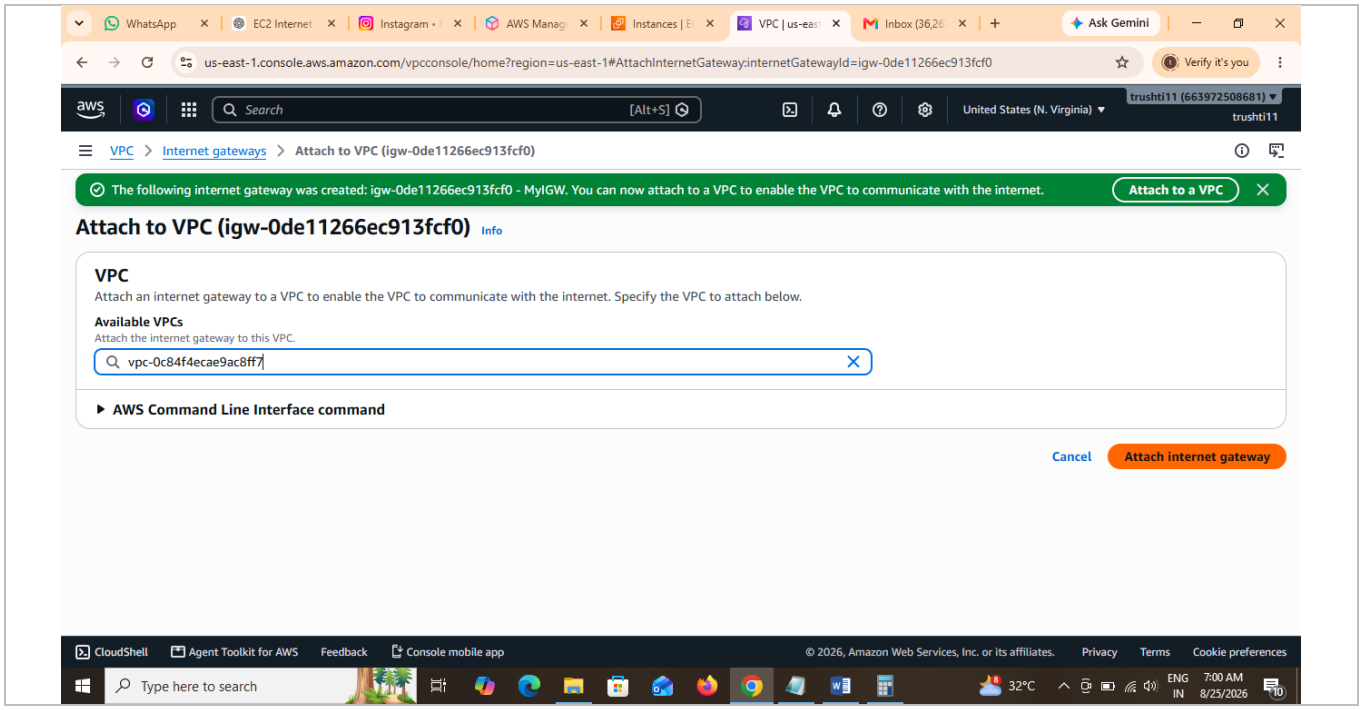

Figure 9: Attaching the internet gateway to my-vpc.

Step 5: Create a Public Route Table and Add an Internet Route

Navigate to Route Tables → Create route table, name it public-RT, and associate it with my-vpc.

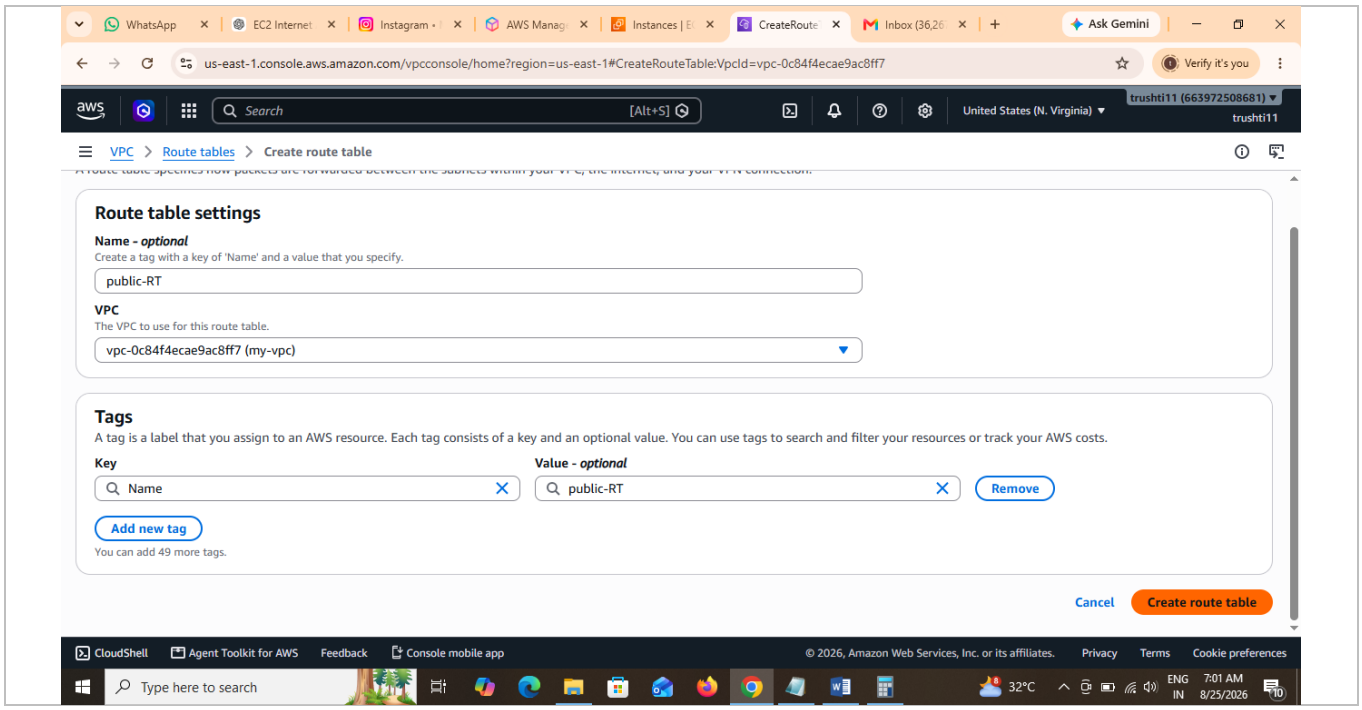

Figure 10: Creating the public-RT route table for my-vpc.

After creation, the route table list shows both the existing Private-RT and the newly created public-RT.

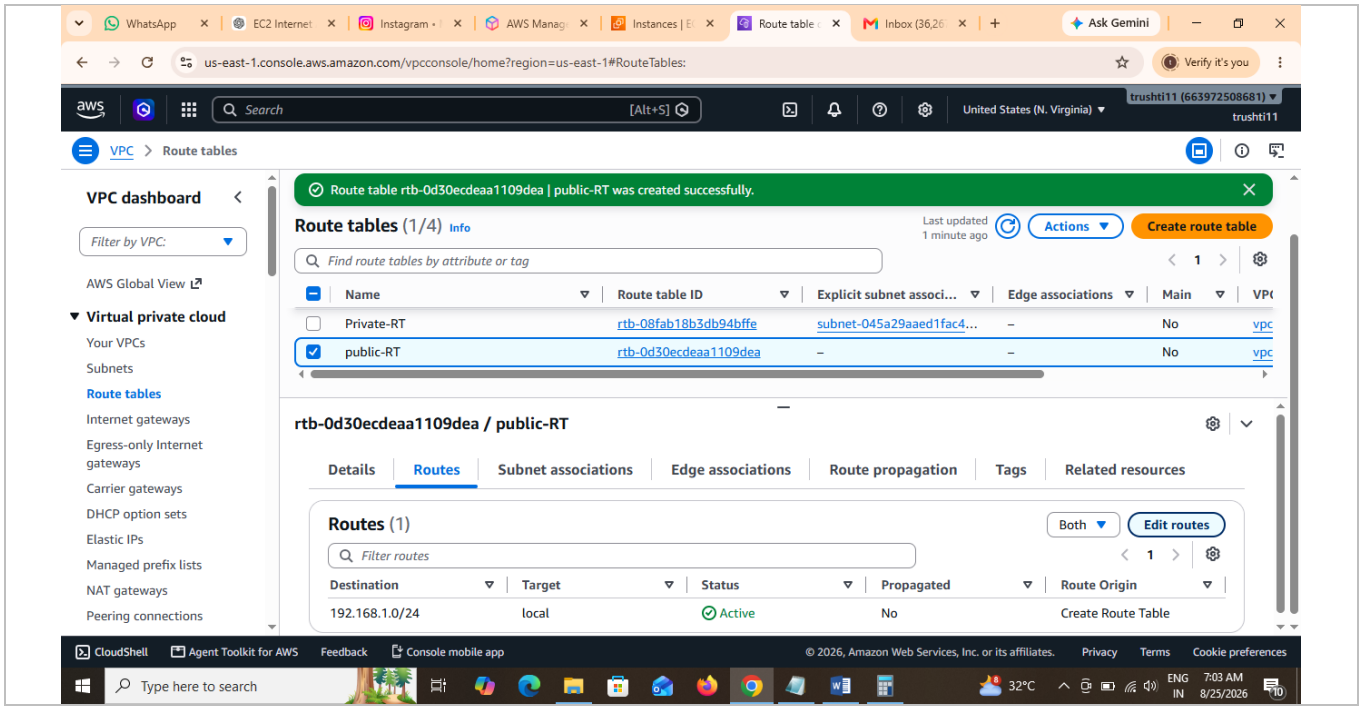

Figure 11: Route tables list showing Private-RT and public-RT.

Open public-RT → Routes → Edit routes, and add a new route with destination 0.0.0.0/0 targeting the internet gateway MyIGW, alongside the default local route for 192.168.1.0/24. Save changes.

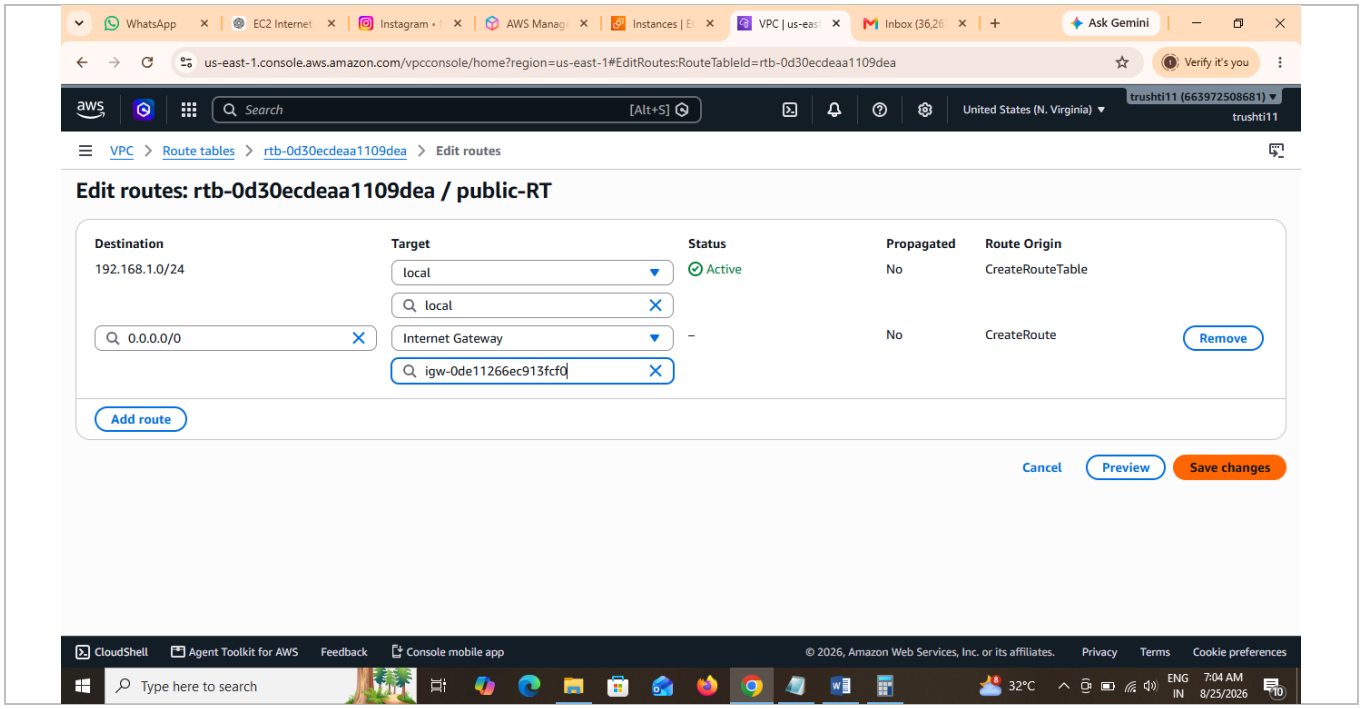

Figure 12: Adding a 0.0.0.0/0 route to the internet gateway in public-RT.

Step 6: Associate the Public Subnet with the Public Route Table

From the route table's Actions menu, choose Edit subnet associations.

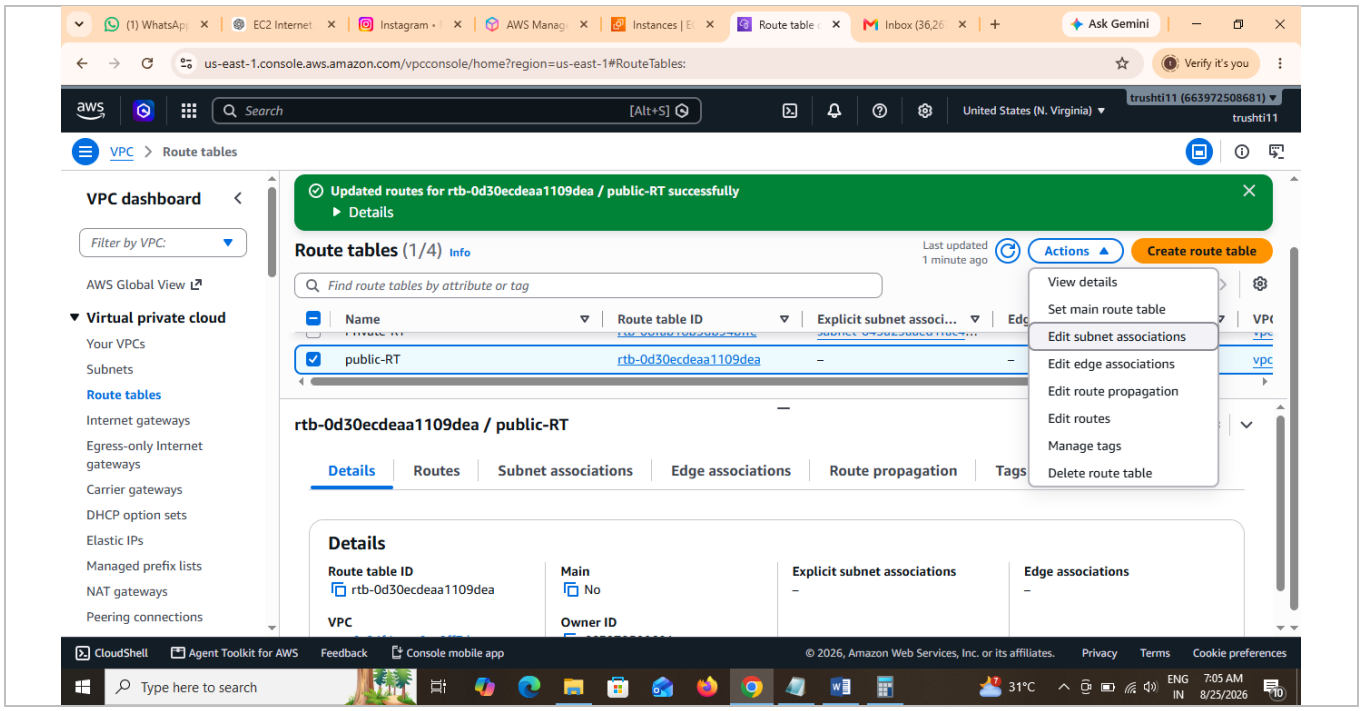

Figure 13: Opening "Edit subnet associations" for public-RT.

Select Public-Subnet from the list of available subnets and save the association, making Public-Subnet a true internet-facing subnet while private-subnet remains tied to Private-RT.

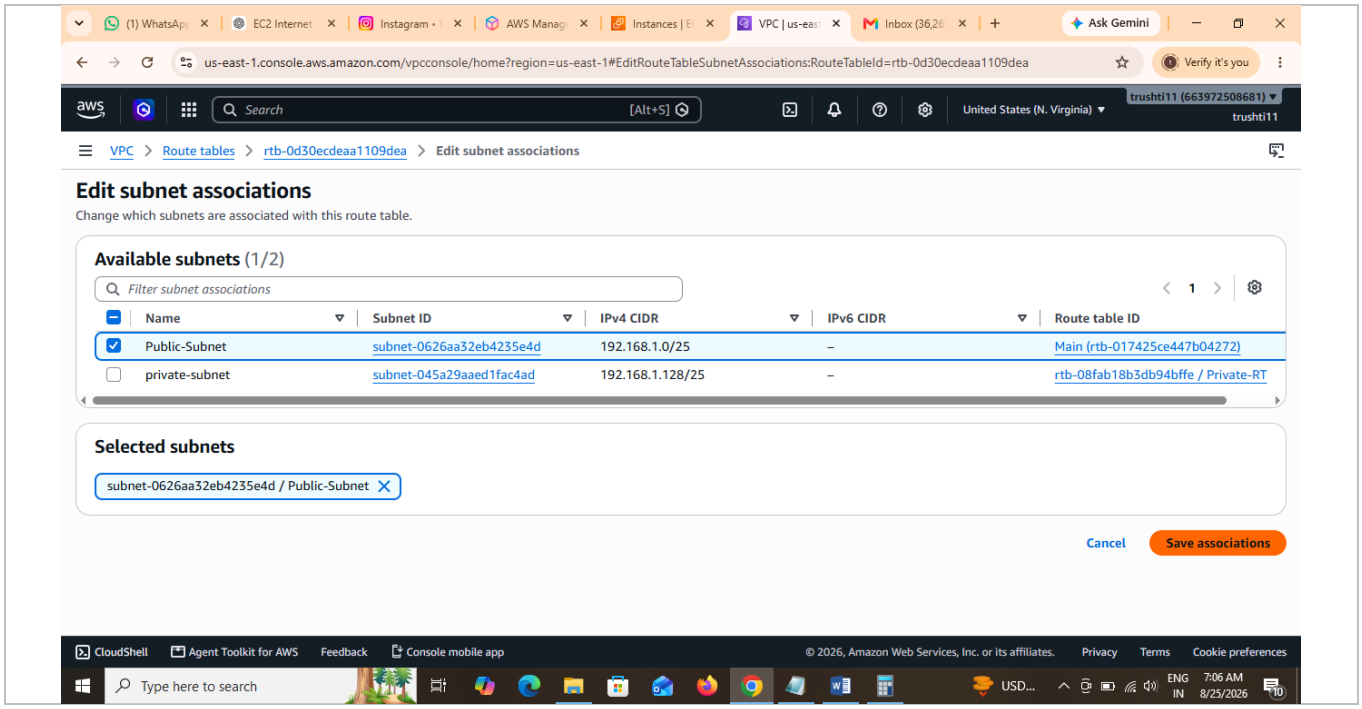

Figure 14: Associating Public-Subnet with public-RT.

Step 7: Launch/Connect to the Public EC2 Instance

A second instance, Public-EC2, is launched in Public-Subnet with a public IP address (13.220.249.203) and the same test1 key pair. From the EC2 console, select Public-EC2 and choose Connect, then pick the "EC2 Instance Connect" (public subnet access) method.

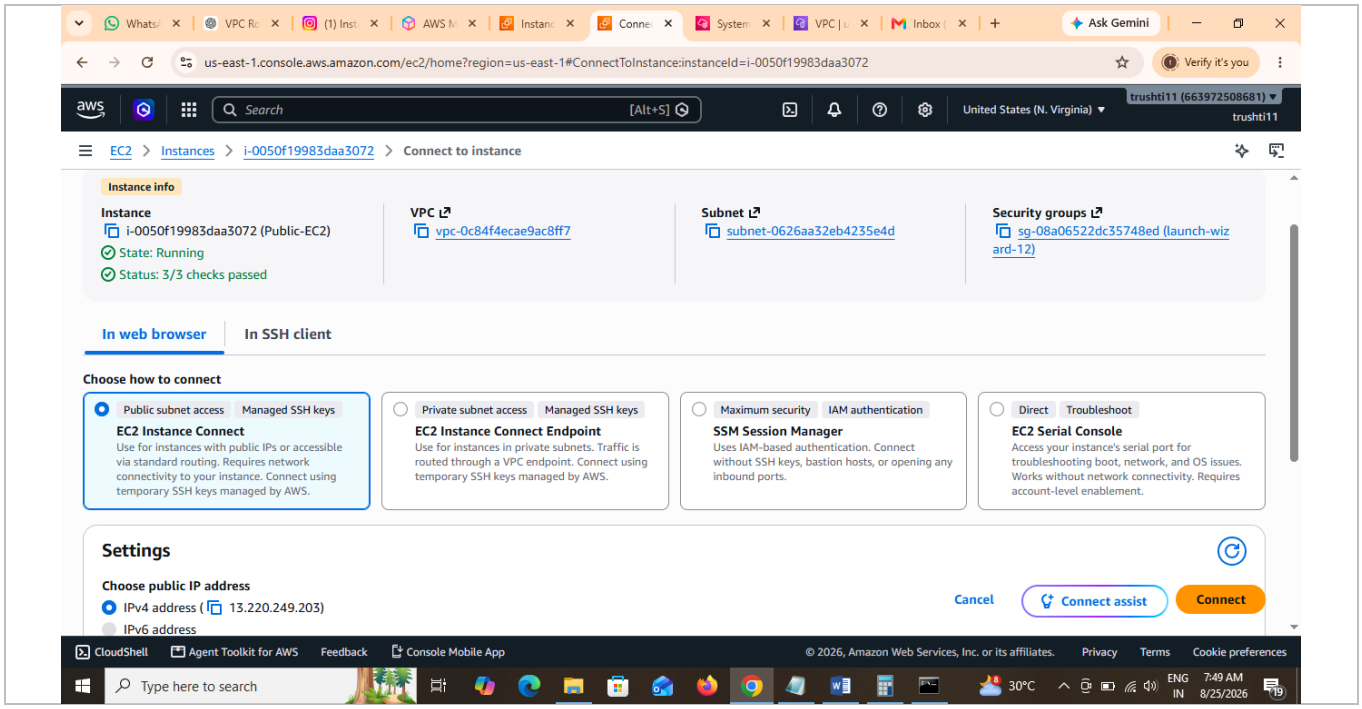

Figure 15: Connect-to-instance options for the public-facing Public-EC2 instance.

Step 8: Copy the Key Pair to the Public EC2 Instance

From the local Windows machine, open Command Prompt and use SCP to securely copy the private key (test1.pem) onto Public-EC2, then SSH into Public-EC2 using the same key:

- scp -i test1.pem test1.pem ubuntu@<Public-EC2-IP>:/home/ubuntu/
- ssh -i test1.pem ubuntu@<Public-EC2-IP>

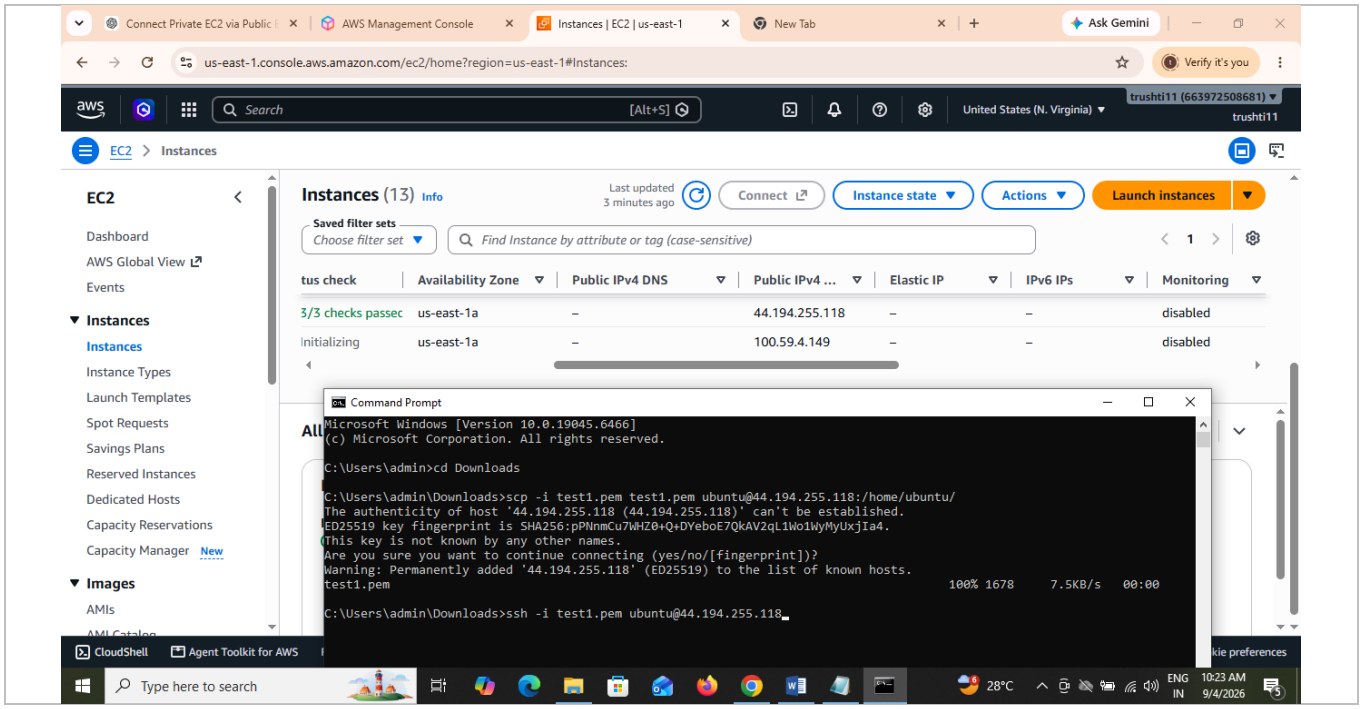

Figure 16: Copying test1.pem to Public-EC2 via SCP and connecting over SSH from the local machine.

Step 9: SSH from the Public EC2 Instance into the Private EC2 Instance

Once logged into Public-EC2, the copied key's permissions are restricted with chmod 400 test1.pem (required by SSH), after which SSH is used to hop into Private-EC2 using its private IP address (192.168.1.191):

- chmod 400 test1.pem
- ssh -i test1.pem ubuntu@192.168.1.191

The system accepts the host's fingerprint on first connection and logs into the private instance successfully.

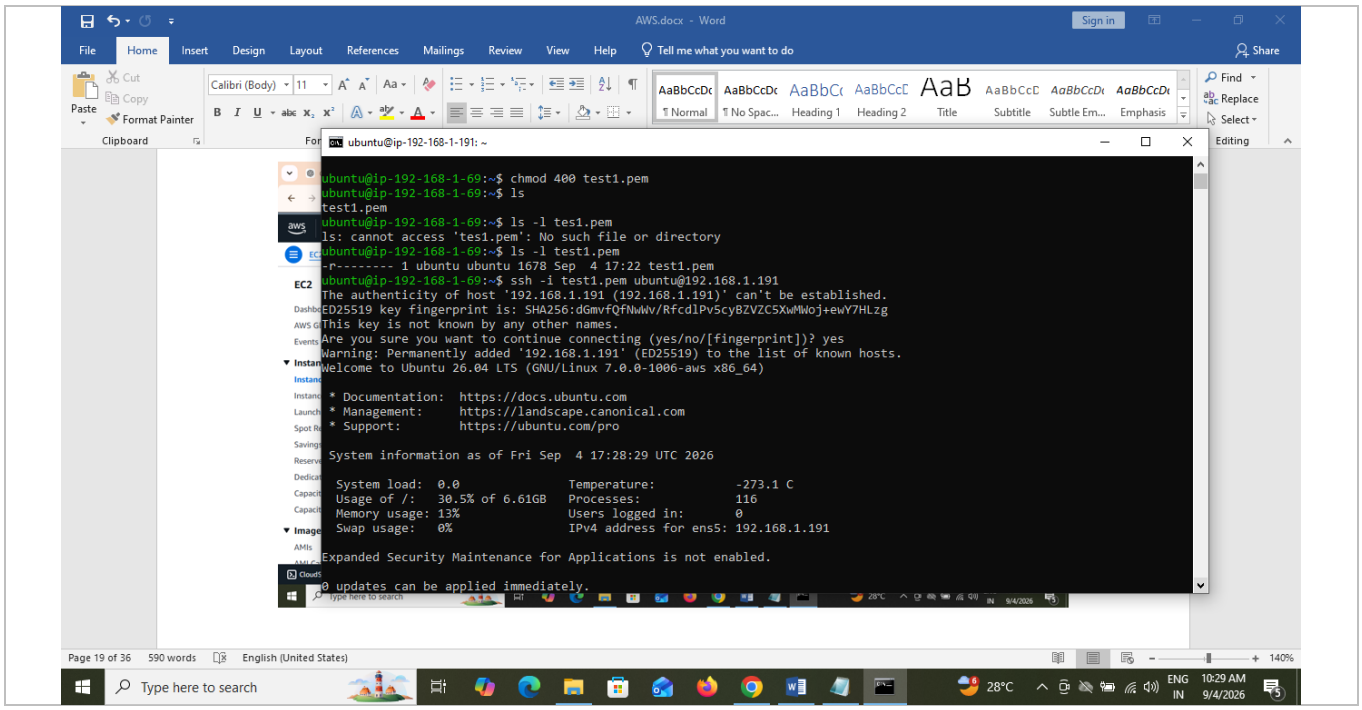

Figure 17: Connecting from Public-EC2 to Private-EC2 (192.168.1.191) over its private IP.

Step 10: Verify the Connection

The shell prompt changes to ubuntu@ip-192-168-1-191, and the system information banner confirms the private IPv4 address (192.168.1.191) of the instance now being controlled — verifying that Private-EC2, which has no public IP and no internet route, has been reached securely through the Public-EC2 bastion host.

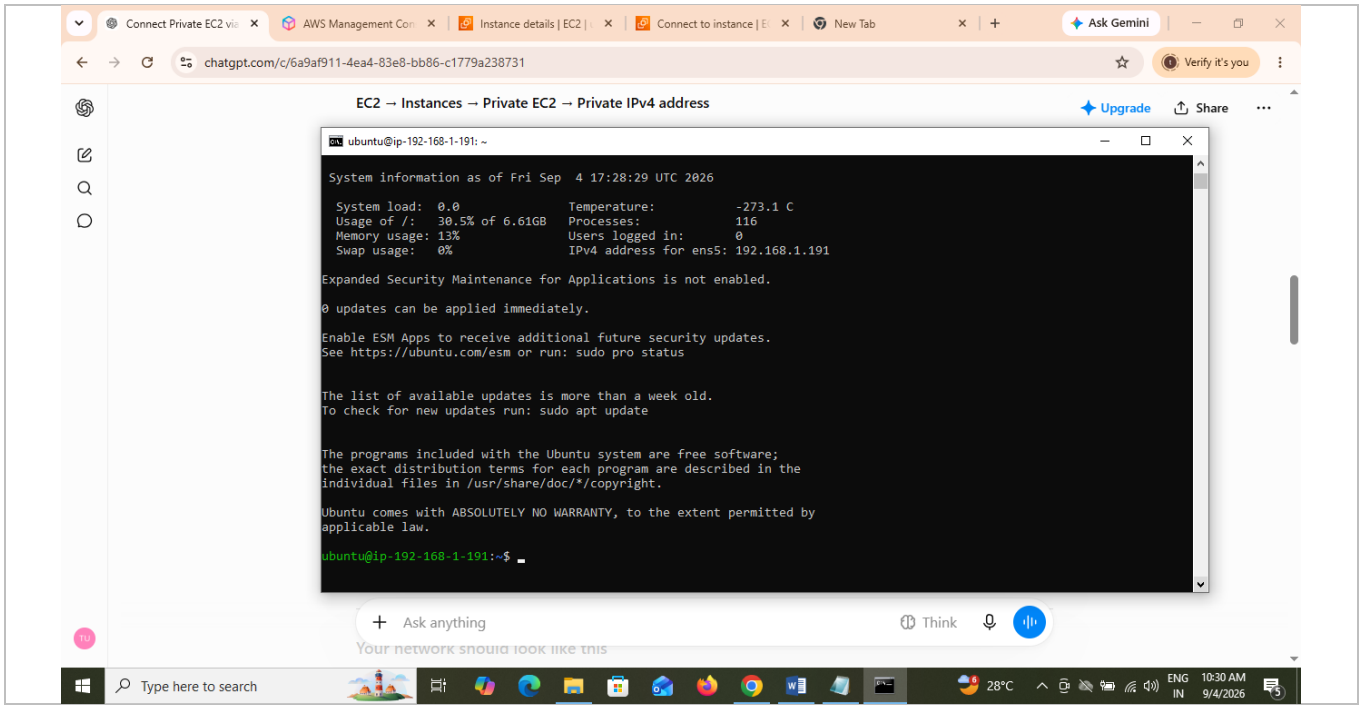

Figure 18: Successful login to Private-EC2, confirmed by its private IP address 192.168.1.191.

5. Conclusion

This lab successfully demonstrated the design and implementation of a secure two-tier AWS network. A custom VPC was segmented into a public and a private subnet, an Internet Gateway and route table were configured to expose only the public subnet to the internet, and an EC2 instance in the private subnet was kept fully isolated from direct internet access.

By configuring a public EC2 instance as a bastion host and relaying SSH access through it using key-pair authentication, secure administrative access to the private instance was achieved without ever assigning it a public IP address — reflecting real-world best practices for securing back-end infrastructure such as databases and internal application servers on AWS.